

AIL7002: Data Structures and Algorithms

Assignment 2: AVL Trees

Overview

In this assignment you will implement a self-balancing AVL tree from scratch. Part 1 covers the AVL tree abstract data type. In Part 2 you will use AVL Trees to solve build a trading desk.

[STARTER CODE](#)

1 Part 1: The AVLNode and AVLTree Classes

AVLNode

To implement an AVL tree you need a type representing a single node, and a class that uses it to build the tree.

```

1 struct AVLNode {
2     int key;
3     int height;           // height of subtree rooted here
4     int subtreeSize;      // number of nodes in subtree rooted here, including
                           // self
5     AVLNode* left;
6     AVLNode* right;
7 };

```

Listing 1: Conceptual node layout

AVLTree Public Interface

Your AVLTree class must expose the following public methods. Signatures are fixed. The supplied AVLTree.h is a pure interface (the “pointer to implementation” idiom) and must not be modified; write your implementation in AVLTree.cpp, placing the node type, the root pointer, and any helpers you need inside struct AVLTree::Impl.

```

1 class AVLTree {
2 public:
3     AVLTree();
4     ~AVLTree();
5
6     bool insert(int key);
7     bool remove(int key);
8     bool contains(int key) const;
9
10    int rank(int key) const;           // # of keys strictly less than 'key'
                                      // currently in the tree,
11                                      // regardless of whether 'key' itself
                                      // is present
12    int select(int k) const;           // the key with rank k (0-indexed); -1
                                      // if k is out of range
13

```

```

14  int height() const;           // height of the tree; -1 for an empty
    tree
15  int size() const;           // number of keys currently stored
16
17  bool isEmpty() const;
18  std::string toString() const; // in-order listing, see format below
19 };

```

Listing 2: Required public interface

Method Semantics

- **insert(key)** — Inserts **key** if absent, rebalancing all ancestors as needed. Returns **true** if the key was newly inserted, **false** if it was already present (tree left unchanged).
- **remove(key)** — Removes **key** if present, rebalancing all ancestors as needed. Returns **true** if key present. **false** otherwise.
- **contains(key)** — Returns **true** iff **key** is present.
- **rank(key)** — Returns the number of keys in the tree strictly less than **key**. This is defined whether or not **key** itself is in the tree (e.g. it is the index at which **key** would be inserted into the sorted key sequence).
- **select(k)** — Returns the key of rank **k** under 0-indexing (i.e. the $(k+1)$ -th smallest key). Returns **-1** if $k < 0$ or $k \geq \text{size}()$.
- **toString()** — Returns the in-order traversal of the keys, formatted exactly as "[1, 2, 3]" — a comma and a single space between keys — or "[]" for an empty tree.

Input and Output Format

Your program reads one test case from standard input and writes its results to standard output. The supplied `main.cpp` does this for you and must not be modified.

The first line holds Q , the number of commands. Each of the next Q lines is one command.

Command	Call	Prints
INSERT k	insert(k)	true or false
REMOVE k	remove(k)	true or false
CONTAINS k	contains(k)	true or false
RANK k	rank(k)	an integer
SELECT k	select(k)	an integer, or -1
HEIGHT	height()	an integer, or -1 on an empty tree
SIZE	size()	an integer
ISEMPTY	isEmpty()	true or false
TOSTRING	toString()	e.g. [1, 2, 3]

Every command prints exactly one line, so the output has Q lines. Booleans print as the lowercase words **true** and **false**.

Constraints. $1 \leq Q \leq 2 \times 10^5$. Keys given to **INSERT**, **REMOVE**, **CONTAINS** and **RANK** satisfy $1 \leq k \leq 10^9$; because keys are positive, the **-1** that **select** returns for an out-of-range index is never a stored key. The argument to **SELECT** is a rank index and may be negative.

Sample cases. `tests/q1/` holds `input_01.txt` through `input_10.txt` with the matching `expected_01.txt` through `expected_10.txt`.

```
make avltree
./avltree < tests/q1/input_01.txt | diff - tests/q1/expected_01.txt
make test-q1      # build and check all ten
```

Worked Example

Each comment is the value the call returns.

```
1 AVLTree t;
2 t.insert(5);      // true
3 t.insert(3);      // true
4 t.insert(8);      // true
5 t.insert(3);      // false -- already present, tree unchanged
6 t.insert(1);      // true
7 t.toString();     // "[1, 3, 5, 8]"
8 t.size();         // 4
9 t.height();       // 2
10 t.contains(8);    // true
11 t.rank(5);        // 2      -- keys 1 and 3 are smaller
12 t.rank(4);        // 2      -- 4 is absent, but rank is still defined
13 t.select(0);      // 1      -- smallest key
14 t.select(3);      // 8      -- largest key
15 t.select(4);      // -1     -- k is out of range
16 t.remove(3);      // true
17 t.remove(7);      // false  -- absent, tree unchanged
18 t.toString();     // "[1, 5, 8]"
19 t.rank(5);        // 1      -- 3 is gone
20 t.height();       // 1      -- the tree rebalanced
```

Listing 3: A sample AVLTree session

That session is the first sample case. It is stored as `tests/q1/input_01.txt`, and the values in the comments are the lines of the matching `expected_01.txt`.

Complexity Requirements

Let n be the number of keys currently stored. The following must all be respected; an implementation that is asymptotically slower than required will lose marks even if it produces correct output.

Operation	Required time complexity
insert, remove, contains	$O(\log n)$
rank, select	$O(\log n)$
height, size, isEmpty	$O(1)$
toString	$O(n)$

2 Part 2: Application — Order Book Engine (Dual-AVL)

In this part you apply the balanced, order-statistic AVL tree from Part 1 to a concrete systems problem. The class to implement is `OrderBookEngine`; its interface is fixed by `OrderBookEngine.h`,

and you write the implementation in `OrderBookEngine.cpp`. As in Part 1, an implementation that is asymptotically slower than required will lose marks even if it produces correct output.

Background

Unusual Trading Inc has a rather unusual trading setup, they trade in only one stock called *Alpha*. Further they only sell the stock and never buy! Analysts working for *Unusual Inc* come up with ”strategies” to determine the best price to sell the stock. A strategy is a set of *orders*, each priced individually, and every order belongs to exactly one strategy.

Every strategy carries a *risk profile*: a single real number summarizing the exposure its pending orders represent. It is not static — it changes each time an order is appended to the strategy but never when an order is executed.

At any time there can be atmost $N = m \times n$ orders, where there are upto m strategies and each strategy can have up to n orders. Your goal is to build a ”Trading Desk” to help *Unusual Inc* achieve their goal. Two major operational events should guide your design.

- **Executing Trades** The desk continuously receives external quotes, each carrying a target limit price. On each quote the engine must locate, across the *entire* pool of pending orders, the one with the highest price that does not exceed the limit. If such an order exists, a trade is executed and the order is immediately removed from the system.
- **Strategy discard.** Strategies are ephemeral — continuously **spun up, re-priced, and pulled** as perceived risk changes. When an analyst discards a strategy, *all* of its pending orders must be invalidated at once, without disturbing any other strategy, and the call must be cheap in the worst case.

The following table describes all the operations that should be supported.

Events

Event	Description	Returns	Target
<code>register_strategy</code>	Register a new strategy together with its opening batch of k orders.	true ; false if the id or the profile is already taken	$O(\log m + k \log N)$
<code>find_safe_strategy</code>	Find the active strategy with the greatest risk profile at or below a given limit.	strategy’s strategy_id ; -1 if none qualifies	$O(\log m)$
<code>place_order</code>	Append one order to an existing strategy, which moves its risk profile.	true ; false on an inactive id	$O(\log m + \log N)$
<code>discard_strategy</code>	The risk kill switch: instantly invalidate a strategy and all its orders.	the discarded strategy_id ; -1 if the range is empty	$O(\log m)$, strict worst case
<code>execute_trade</code>	Execute the highest-priced order at or below the limit whose strategy is still active, and take it off the book.	order price; -1 if nothing is eligible	$O(\log N)$
<code>mock_trade</code>	Report the price execute_trade would return right now, without changing anything.	order price; -1 if nothing is eligible	$O(\log N)$

Note that N can become larger than total active orders if we implement $O(\log m)$ deletion.

Guarantees

The driver is guaranteed to respect the following. Your implementation may rely on them; the reference behaviour if one is violated is stated in the semantics below.

- `strategy_id` values are positive and are never reused over the whole run.
- At any instant, the active strategies have pairwise-distinct risk profiles.
- At any instant, the pending orders have pairwise-distinct prices.
- The range passed to `discard_strategy` holds at most one active strategy.

Public Interface: OrderBookEngine

The function signatures are fixed. Implement them in `OrderBookEngine.cpp`; `OrderBookEngine.h` is a pure interface and must not be modified. All data members and helper functions must be defined within `struct OrderBookEngine::Impl` in the `.cpp` file.

```

1  class OrderBookEngine {
2  public:
3      OrderBookEngine();
4      ~OrderBookEngine();
5
6      // Register 'strategy_id' with risk profile 'risk' and an opening
7      // batch of orders.
8      bool register_strategy(int strategy_id, double risk,
9                             const std::vector<long long>& prices);
10
11     // Active strategy with the greatest risk profile <= max_risk; -1 if
12     // none.
13     int find_safe_strategy(double max_risk) const;
14
15     // Append one order at 'price'; the engine moves the strategy's
16     // profile itself.
17     bool place_order(int strategy_id, long long price);
18
19     // Kill switch: deactivate the one strategy whose profile is in [
20     // low_risk, high_risk].
21     int discard_strategy(double low_risk, double high_risk);
22
23     // Execute the highest still-valid pending price <= limit_price; -1 if
24     // none.
25     long long execute_trade(long long limit_price);
26
27     // Report what execute_trade would fill, without executing it; -1 if
28     // none.
29     long long mock_trade(long long limit_price);
30 };

```

Listing 4: Required public interface

Method Semantics

- `register_strategy(strategy_id, risk, prices)` — Registers a new strategy whose opening risk profile is `risk`, and adds each price in `prices` as a pending order corresponding to that strategy. The batch may be empty. **Do not recompute the risk here.** Returns `false` and makes no changes if `strategy_id` is already active, no orders from the batch are added in this case. Otherwise, returns `true`.
- `find_safe_strategy(max_risk)` — Returns the `strategy_id` of the active strategy whose risk profile is the greatest value not exceeding `max_risk`. Returns `-1` if no active strategy qualifies.
- `place_order(strategy_id, price)` — Appends one pending order at `price` to the already-registered `strategy_id` and adds it to the price index. Appending an order moves the strategy's risk profile, use `update_risk(price, current_risk)`, to get the updated risk. Returns `false` and changes nothing if `strategy_id` is not active, or in the rare case where the new profile coincides with another active strategy's; returns `true` otherwise.
- `discard_strategy(low_risk, high_risk)` — Discards the active strategy whose risk profile lies in the closed range `[low_risk, high_risk]` and invalidates all of its pending orders. Returns the strategy's `strategy_id`, or `-1` if no active strategy has a risk profile in the range. Both endpoints are inclusive, and the range is guaranteed to contain at most one active strategy, so the target is always unambiguous. The operation must run in $O(\log m)$ in the *strict* worst case; in particular, it may not iterate over the strategy's orders.
- `execute_trade(limit_price)` — Finds the highest-priced pending order with price $\leq$ `limit_price` whose owning strategy is active, executes the order by removing it from the pool, and returns its price. Returns `-1` if no eligible order exists.
- `mock_trade(limit_price)` — Returns the price that `execute_trade(limit_price)` would return if it were called right now, or `-1` if it would find nothing **without deleting the order**.

Input and Output Format

Your program reads one test case from standard input and writes its results to standard output. The supplied `main_orderbook.cpp` does this for you and must not be modified.

The first line holds Q , the number of commands. Each of the next Q lines is one command.

Command	Call	Prints
REGISTER s V k P_1 ... P_k	<code>register_strategy(s, V, {P1..Pk})</code>	<code>true</code> or <code>false</code>
PLACE s P	<code>place_order(s, P)</code>	<code>true</code> or <code>false</code>
FIND_SAFE R	<code>find_safe_strategy(R)</code>	a strategy id, or <code>-1</code>
DISCARD L H	<code>discard_strategy(L, H)</code>	a strategy id, or <code>-1</code>
EXECUTE C	<code>execute_trade(C)</code>	an order price, or <code>-1</code>
MOCK C	<code>mock_trade(C)</code>	an order price, or <code>-1</code>

On a REGISTER line the count k precedes the k prices, and k may be zero. Every command prints exactly one line, so the output has Q lines; booleans print as the lowercase words `true` and `false`. Each line carries exactly the arguments its call takes, so the driver only reads tokens and prints return values. Risk profiles never appear in the feed beyond the opening V on a REGISTER line: from then on the engine maintains each strategy's profile itself, through `update_risk()`.

Constraints. $1 \leq Q \leq 2 \times 10^5$. Strategy ids and order prices satisfy $1 \leq s, P \leq 10^9$; because prices are positive, the -1 returned by `execute_trade` and `mock_trade` is never a real price. Risk profiles, and the arguments to `FIND_SAFE` and `DISCARD`, are real numbers — read them as `double`.

Sample cases. `tests/q2/` holds `input_01.txt` through `input_10.txt` with the matching `expected_01.txt` through `expected_10.txt`.

```
make orderbook
./orderbook < tests/q2/input_01.txt | diff - tests/q2/expected_01.txt
make test-q2          # build and check all ten
```

Worked Example

Below is the first sample case, `tests/q2/input_01.txt`. Everything from the bar onwards is annotation: the line `expected_01.txt` holds for that command, and then why.

```
16
REGISTER 1 10.0 2 10 30 | true  s1 opens at 10.0 owning orders 10, 30
REGISTER 2 20.0 2 20 50 | true  s2 opens at 20.0 owning orders 20, 50
FIND_SAFE 15.0          | 1     10.0 is the greatest profile <= 15.0
FIND_SAFE 25.0          | 2     20.0 fits too, and is greater
FIND_SAFE 5.0           | -1    no active strategy is that safe
MOCK 35                 | 30    best price <= 35; nothing is filled
EXECUTE 35               | 30    the same order, now filled and gone
MOCK 35                 | 20    30 is gone, so 20 is next in line
PLACE 1 5               | true  s1 appends 5; profile 10.0 -> 13.0
FIND_SAFE 15.0          | 1     s1 re-keyed to 13.0, still <= 15.0
DISCARD 12.0 14.0       | 1     the one strategy in that range is s1
FIND_SAFE 15.0          | -1    s1 is gone, and s2 sits at 20.0
MOCK 100                | 50    s2's orders survived the discard
EXECUTE 100              | 50
EXECUTE 100              | 20
EXECUTE 100              | -1    s1's stale 10 and 5 are all that is left
```

The one derived number is on the `PLACE` line: `update_risk(5, 10.0)` is 13.0, which is why the `DISCARD` range `[12.0, 14.0]` then catches `s1`. Orders 10 and 5 outlive their strategy and are dropped lazily, by the first search that reaches them — the final `EXECUTE`.

Complexity Requirements

Let m be the number of currently active strategies and N the number of currently pending orders.

Operation	Required time complexity
<code>register_strategy</code> (batch of k orders)	$O(\log m + k \log N)$
<code>find_safe_strategy</code>	$O(\log m)$
<code>place_order</code>	$O(\log m + \log N)$
<code>discard_strategy</code>	$O(\log m)$, strict worst case
<code>execute_trade</code>	$O(\log N)$
<code>mock_trade</code>	$O(\log N)$

What is being provided?

The assignment packet contains the starter code for both parts along with sample test cases.

1. `AVLTree.h`: pure-interface header for the `AVLTree` class described in Part 1. (**You should not modify this file.**)
2. `AVLTree.cpp`: here you implement every method of `AVLTree`, including `struct AVLTree::Impl` and the node type. The stubs compile and run as supplied, but each returns a placeholder, so an untouched checkout gives wrong answers rather than an error.
3. `main.cpp`: the Part 1 driver. It reads one test case from standard input, calls your `AVLTree`, and writes the results to standard output. (**You should not modify this file.**)
4. `OrderBookEngine.h`: pure-interface header for the `OrderBookEngine` class described in Part 2. (**You should not modify this file.**)
5. `OrderBookEngine.cpp`: here you implement every method of `OrderBookEngine`, including `struct OrderBookEngine::Impl`. This file also supplies `update_risk()`, which **must not be modified**.
6. `RiskProfile.cpp` and `RiskProfile.h` containing the function `update_risk` required for adding new order to existing strategy. (**You should not modify this file.**)
7. `main_orderbook.cpp`: the Part 2 driver. It reads one test case from standard input, calls your `OrderBookEngine`, and writes the results to standard output. (**You should not modify this file.**)
8. `Makefile`: builds and runs both parts.
9. `tests/q1/` and `tests/q2/`: ten sample cases per part. Every `input_NN.txt` has a matching `expected_NN.txt` holding the output a correct implementation produces for it.

The two parts build independently. `OrderBookEngine` does not link against your Part 1 `AVLTree`: the trees it needs carry a value alongside each key and answer predecessor queries, so you write them inside `OrderBookEngine.cpp`. Starting from your Part 1 code and generalising it is a reasonable way to go about that.

How to run?

`make` builds both parts; `make avltree` and `make orderbook` build one at a time. Each program reads a single test case on standard input and writes its answers to standard output, so you run a case by redirecting a file into it.

```
make
./avltree    < tests/q1/input_01.txt
./orderbook < tests/q2/input_01.txt
```

To check a case rather than read it, pipe the output through `diff` against the expected file. `make test-q1` and `make test-q2` do that for all ten cases of a part and print one PASS or FAIL line per case; `make test` runs both parts.

```
./avltree < tests/q1/input_01.txt | diff - tests/q1/expected_01.txt
make test
```

What to submit?

1. Submit a single zip file named `<your-entry-number>.zip`.
2. On extraction it must yield one folder named `<your-entry-number>`.
3. That folder holds your source files and a `writeup.txt`. The source files are `AVLTree.h`, `AVLTree.cpp`, `OrderBookEngine.h`, `OrderBookEngine.cpp`, and any additional header you wrote. Submit `.h` and `.cpp` files only — no binaries, no object files, no test data.
4. `writeup.txt` must open with a line naming everyone you discussed or collaborated with (see the guidelines on collaboration versus cheating on the course home page). If you never discussed the assignment with anybody, write “None”. After that you are welcome to add comments about your submission, though this is not necessary.
5. For example, entry number 2025AIB0001 submits `2025AIB0001.zip`, which extracts to the following structure:

```
2025AIB0001
|
---- AVLTree.h
---- AVLTree.cpp
---- OrderBookEngine.h
---- OrderBookEngine.cpp
---- main_orderbook.cpp
---- writeup.txt
```

6. Please compress your directory structure into a `.zip` file only.
7. You will be penalized for any submission that does not conform to the above requirements.

Evaluation Criteria

The assignment is worth 8 points. Your code will be autograded against a series of tests. 5 points are for autograding and 3 points are for the demo with the TA.

What is allowed? What is not?

1. This is an individual assignment.
2. Your code must be your own. You are not to take guidance from any general purpose code or problem-specific code meant to solve these or related problems.
3. You are not allowed to use built-in (or anyone else’s) implementations of balanced trees or order-statistic structures. A key aspect of the course is to have you learn how to implement these data structures. In particular, do not use `std::set`, `std::multiset`, `std::map`, `std::multimap`, or any equivalent self-balancing container to do the work of your trees. Hash containers — `std::unordered_map` and `std::unordered_set`.
4. You should develop your algorithm using your own efforts. You should not Google search or use AI tools for direct solutions to this assignment. However, you are welcome to Google search for generic C++-related syntax.

5. You must not discuss this assignment with anyone outside the class. Make sure you mention the names in your write-up in case you discuss with anyone from within the class. Please read academic integrity guidelines on the course home page and follow them carefully.
6. Your submitted code will be automatically evaluated against another set of benchmark problems. You get a significant penalty if your output is not automatically parsable and does not follow the input-output guidelines.
7. We will run plagiarism detection software. Anyone found guilty will be awarded a suitable penalty as per course guidelines, and institute rules.
8. As per the LLM policy of the class, no code should be written by an LLM. However, you are allowed to use an LLM to understand C++ syntax, and also to brainstorm ideas related to the assignment.